Instance States and Continuous Normalization in TeSpec

Operational semantics and implementation correspondence

Implementation note

October 10, 2026

1 Problem and semantic scope

TeSpec tests a partially instantiated separation-logic contract by symbolic execution. It supports two control-flow policies: execution of loop iterations and available function bodies (`expand`), and QCP's invariant and contract abstraction (`no-expand`). Both policies maintain current instance information at program points and evaluate known logical expressions. The distinction concerns control-flow abstraction, not whether executable logical definitions may be unfolded.

Let $\sigma = (\rho, h, \eta)$ consist of a C variable environment, a heap, and a logical valuation. We use QCP's interpretation of typed storage, structure layout, arithmetic, and separating conjunction. In $A * B$, the two assertions hold on disjoint component heaps. In $A \wedge B$, they hold on the same heap.

A contract is $\{R(\bar{a}_0, \bar{w})\}\ f(\bar{a})\ \{E(\bar{a}_0, \bar{w}, r)\}$. The tuple $\bar{a}_0$ denotes immutable entry snapshots and $\bar{w}$ denotes contract logical parameters. A typed partial input substitution θ_0 fixes selected snapshots and logical parameters. Pointer values may remain symbolic while logical heap contents are fixed.

When a substitution occurs as a proposition, it denotes its graph: $\bigwedge_{x \mapsto v \in \theta_0} x = v$. Two queries must be distinguished:

$$\exists \eta, h.\ (\eta, h) \models R[\theta_0], \tag{1}$$

$$\{R[\theta_0]\}\ \mathit{body}\ \{E[\theta_0]\}. \tag{2}$$

An existential witness for (1) is not installed as an execution input for (2). Unbound inputs in the latter range over all interpretations satisfying the instantiated precondition.

2 Product states

Definition 1 (Test state)**.** *A branch at program point ℓ carries*

$$T = \langle \ell, P, I, \theta_0, \Omega, m \rangle, \qquad I = \langle \nu, L, K, M \rangle.$$

P is the current QCP assertion, I is the current instance view, Ω is the set of outstanding obligations, and m is the execution mode. L maps program-variable identities to symbolic storage locations, K contains valid logical equalities, and M records typed storage and spatial predicates. ν identifies an observation version.

The instance view is branch-local and owns its data. Its heap component uses QCP's existing spatial representation; it does not allocate numeric addresses or introduce a second concrete heap semantics. Opaque predicates may remain symbolic entries in M.

Write $\gamma(I)$ for the consistency relation represented by an instance view. The conceptual product denotes

$$[\![T]\!] = \{\sigma \mid \sigma \models P \wedge \gamma(I) \wedge \theta_0\}.$$

The implementation uses a materialized representation: valid instance refinements are included in the assertion before a cut, and synchronized views satisfy $P \models \gamma(I)$. In particular, the product is not encoded as $P * \gamma(I)$, which would claim a second copy of the same storage.

Reconstructing I from a weakened assertion alone loses information. Consequently, an annotation cut first transports available instance bindings to its target, generates the corresponding obligation, and only then captures the successor view. Ordinary execution rebuilds the view from QCP's updated formula; it never replays entry heap contents over a modified heap.

3 Instantiation and evaluation

Definition 2 (Logical instantiation)**.** *$F[\theta]$ is typed, capture-avoiding substitution by logical identity. It removes instantiated existential binders and freshens independent binders when needed. A symbolic pointer substituted for an existential is a witness term, not a request to rename the existential binder to that pointer.*

Entry snapshots, current C values, and callee post-state variables have distinct identities even when their source names coincide. Define point instantiation by

$$\mathsf{Inst}_T(F) = \mathsf{N}\big(\mathsf{Resolve}(F[\theta_0]; L, K, M)\big).$$

Resolution uses the current storage and valuation, with QCP's obligations for memory reads and C operations. Normalization evaluates known arithmetic, Boolean, list, and constructor operations, and unfolds accessible definitions under a budget. Unknown subexpressions remain symbolic.

For a valid branch context Γ, the required normalization property is

$$\theta_0 \wedge \Gamma \models F \leftrightarrow \mathsf{N}_\Gamma(F).$$

Fresh values introduced by a definition expansion are existentially closed on the right. C safety conditions remain obligations; mathematical simplification must not erase overflow, division-by-zero, invalid-shift, or storage conditions.

For example, $n_0 = 2$ and $xs_0 = [10, 20]$ describe the entry instance. After `n++; p[0]=n;`, the current value is $n = 3$ and current contents are $[3, 20]$, while xs_0 remains $[10, 20]$.

4 Common operational rules

For an ordinary statement c, the common transition is

$$\frac{\widehat{P} = \mathsf{Materialize}(P, I, \theta_0) \quad (P_r, V) = \mathsf{QCPstep}(c, \widehat{P}) \quad P' = \mathsf{N}(P_r) \quad I' = \mathsf{Capture}(P')}{\langle \ell, P, I, \theta_0, \Omega, m\rangle \longrightarrow_c \langle \ell', P', I', \theta_0, \Omega \cup V, m\rangle}.$$

Materialization is normalization when instance refinements are already present in the formula. Assignment evaluates the right side and updates the QCP storage for the left side. A write through q must update the observation through p when the state establishes $p = q$. Previous storage contents cannot remain current facts merely because the pointer identities did not change.

Branches produce separately owned views for $P \wedge b$ and $P \wedge \neg b$. Unknown branches remain; only justified contradictions can prune a branch. A join either preserves a disjunction or retains common facts. An instance substitution from one branch cannot be exported to a sibling.

The implementation refreshes at statement and expression boundaries, after predicate expansion and contract application, and before consuming instance information. Substitution, merge, and principal write interfaces invalidate old views. Query-time refresh covers QCP strategies that modify assertions in place. This is a whole-view invalidation scheme; ν is not a per-cell SSA version or a heap address.

5 Assertion cuts and intermediate instantiation

For an ordinary assertion A, let

$$\theta_c = \mathsf{Match}_{QCP}(P, I, A), \qquad A^* = \mathsf{N}(A[\theta_0][\theta_c] \wedge \mathsf{Pure}(P)).$$

The cut emits $P \vdash A^*$ and continues with $(A^*, \mathsf{Capture}(A^*))$. Matching provides a candidate, not a certificate. A well-scoped substitution for target existentials satisfies $A^* \vdash A[\theta_0]$ by existential introduction. A pending entailment remains in Ω and prevents an unconditional successful result.

The matcher reuses QCP's representation and matching rules for cells, arrays, fields, spatial predicates, and logical value terms. It specializes each incoming path separately. A source location reached with a new instance does not reuse the first visit's candidate substitution.

For example,

$$P = x \mapsto 3, \quad A = \exists v.\ x \mapsto v \wedge v \geq 0 \quad \Rightarrow \quad \theta_c = [v \mapsto 3], \quad A^* = x \mapsto 3.$$

After an increment, a subsequent cut must carry 4, not 3. Pure path facts survive annotation processing because an annotation does not execute a write. They refer to logical value identities, with current storage related to those identities by matching.

If matching cannot recover a relation, the backend retains symbolic terms and proof obligations. Opaque definitions and abstractions that omit a content relation can therefore remain precision barriers. A complete per-point view may contain unknown values; it is not a promise of one closed numeric state.

Which-implies annotations use the same proof-only refinement principle. After QCP applies the local specification, the current incoming instance specializes its output. A nontrivial substitution emits an additional entailment from the original state to that refined output. The original which-implies obligations remain. This transfer is not applied to mutating function calls.

In expand mode, `ignore` leaves proof annotations out of state abstraction. Under `check-preserve`, annotations are checked on a copy, obligations are retained, and the actual execution state continues unchanged.

6 Function calls

In no-expand mode, arguments are evaluated in the current product state. QCP freshens the contract and matches its precondition, obtaining θ_g and frame F. The obligations and successor have the form

$$P \vdash R_g[\theta_g] * F, \qquad P' = \mathsf{N}(E_g[\theta_g] * F), \qquad I' = \mathsf{Capture}(P').$$

Postcondition existentials are fresh. Old contents in the modified region do not survive unless the postcondition relates them to the new contents. Frame facts do survive. A contract with $p \mapsto v$ and $p \mapsto v + 1$ transports input 3 to output 4; $\exists w.\ p \mapsto w$ leaves the output symbolic.

Expand mode creates a fresh frame and executes the available body, continuously updating (P, I). The return transfers the actual heap changes and value, and removes local storage. A declaration-only call uses its contract and records that abstraction. A call with neither body nor contract is an error. A bodyless test target performs precondition checking only; its execution safety and return-condition phases are not run.

7 Loops

Expand mode evaluates the guard and body repeatedly, producing versions (P_k, I_k) for successive visits. A symbolic guard may fork. Break, continue, and return transfer their corresponding branch states. Exhausting a bound reports an incomplete analysis.

No-expand mode has three distinct instance environments:

$$\begin{aligned} &P_{\text{entry}} \vdash J[\theta_0], && I_{\text{entry}} = \mathsf{Capture}(P_{\text{entry}}),\\ &\{J[\theta_0] \wedge b\}\ \mathit{body}\ \{J[\theta_0]\}, && I_{\text{ind}} = \mathsf{Capture}(J[\theta_0] \wedge b),\\ &P_{\text{exit}} = \mathsf{N}(J[\theta_0] \wedge \neg b), && I_{\text{exit}} = \mathsf{Capture}(P_{\text{exit}}). \end{aligned}$$

The induction environment denotes an arbitrary invariant-satisfying iteration. An entry fact $i = 0$ cannot specialize the induction variable to zero. With $n_0 = 2$ and invariant $0 \le i \le n_0$, the induction range is $0 \le i \le 2$; the exit condition $i \ge 2$ recovers $i = 2$.

Even a fully specified input can yield symbolic intermediate values in this mode: a loop point summarizes several iterations, and a weak invariant or contract can admit several outcomes. Preserving instance information means preserving justified relations, not selecting an arbitrary outcome.

8 Satisfiability and outcome discipline

Precondition checking operates on its own normalized copy of $R[\theta_0]$. It combines accessible predicate definitions, pure conditions, storage ranges, layout, address bounds, and separation constraints. An UNSAT result for necessary conditions excludes the original precondition. SAT for necessary conditions alone does not establish (1); a complete encoding or a checked complete witness is required. Residual definitions retain the original existence obligation for interactive proof.

Initial UNSAT is an invalid instance. Unclosed existence or execution obligations produce UNKNOWN and residual VCs. Auto/manual classifies whether the automatic backend closed a VC; replayable certificate status is recorded separately. A failed entailment after abstraction is not by itself a reachable execution counterexample.

9 Relative correctness argument

Proposition 1 (Synchronization and execution preservation)**.** *Assume the underlying QCP transitions are sound, normalization preserves meaning, and substitutions respect types and binder identities. For a valid*

synchronized predecessor, a transition satisfying the rules above produces a successor whose current instance observations are supported by its formula. When the accumulated obligations hold, concrete executions represented by the predecessor are covered by the successor representation.

Argument by transition class. Ordinary steps inherit QCP's transition property, and capture observes the result. Normalization preserves denotation. Assertion cuts check the refined target itself, so the carried relations are justified by the same obligation. Branches retain separate valuations, and joins use disjunction or common facts. Calls use the frame rule and fresh post-state variables. Loop induction uses an independent invariant state; expansion composes individual execution steps. Budget exhaustion never supplies a success premise. □

This is a relative semantic argument and an implementation contract. The C modules have executable regression and sanitizer coverage; a machine-checked proof of their implementation is a separate artifact.

10 Implementation and reproducibility

Component	sac location (under `SymExec`)
Owned per-assertion view	`AsrtDef/AssDef.c`
Input and current contexts	`SymExec/InstanceContext.c`
Shared normalization	`Utility/ExpressionNormalize.c`
Statement and cut transitions	`middle_end/SymExecutor.c`
Contract/frame transfer	`SymExec/FuncCall.c`
Optional point observations	`SymExec/SpecTest.c`
Structured output	`JsonPrinter/JsonFormat.c`

Use `-instance-trace -Json-output` to record statement and condition-expression boundary formulas, memory observations, equalities, source locations, branches, and versions. The default artifacts omit the full trace. The new regression suite is `SymExec/tests/run-instance-state-tests.py`; it checks positive cases and stale-value, alias, branch, contract, and induction counterexamples in both modes. Expected UNKNOWN outcomes establish rejection of an incorrect automatic success, not a completed proof of the residual goal. Broad example smoke results are reported separately from semantic expectations.